

Attention ① 관계를 계산하는 방법

Q·K·V와 scaled dot-product attention을 손으로 계산한다.

causal mask와 텐서 shape를 검증한다.

단일 헤드 문자 언어 모델을 학습하고 생성한다.

TOKENS
→ **RELATIONS**

00-02분 · 이번 주에 만들 결과물 확인

90분 수업 지도

- 01 00-05 도입 · 05-20 개념 · 20-35 수식과 손계산
- 02 35-50 구현 설계 · 50-75 실습 · 75-85 분석
- 03 85-90 회고 · 부록과 자율 과제는 수업 이후

90 MIN / 개념 45 + 실습 25 + 분석·회고 20

02-05분 · 실습 노트북 열기

언어 모델의 과제

the ca

다음 토큰 분포

t 샘플링

the cat

- 01 앞의 토큰을 보고 다음 토큰의 분포를 예측한다.
- 02 학습 입력과 정답은 한 칸 어긋난 시퀀스다.
- 03 생성은 예측한 토큰을 문맥 뒤에 붙이며 반복한다.

05-20분

문자를 정수로 바꾸기

- 01 train 문자 집합으로 vocabulary를 만든다.
- 02 각 문자를 ID로 바꾸고 embedding에서 벡터를 찾는다.
- 03 모르는 문자를 위한 UNK를 둔다.

문자 → token ID → 학습 가능한 벡터

05-20분

고정 문맥의 한계

- 01 bigram은 바로 이전 문자에만 조건부로 의존한다.
- 02 문장 의미는 더 먼 위치에도 의존할 수 있다.
- 03 어텐션은 문맥 안의 여러 위치를 가중합한다.

$$p(x_t | x_{<t}) \text{ 와 } p(x_t | x_{(t-1)})$$

05-20분

Q·K·V의 역할

X

$$Q=XW_Q / K=XW_K \\ / V=XW_V$$

유사도

가중합

- 01 Query는 현재 위치가 찾는 관계의 기준이다.
- 02 Key는 각 위치가 비교에 내놓는 표현이다.
- 03 Value는 가중합으로 전달할 내용이다.

05-20분

self와 cross attention

- 01 self-attention은 Q·K·V가 같은 시퀀스에서 나온다.
- 02 cross-attention은 Q와 K·V의 출처가 다르다.
- 03 오늘은 decoder 방식의 causal self-attention을 만든다.

self: $X \rightarrow Q, K, V$ / cross: $X \rightarrow Q, Y \rightarrow K, V$

05-20분

미래를 보지 않는 규칙

점수 행렬

상삼각 차단

행별 softmax

과거 정보 혼합

- 01 위치 t 는 위치 t 이하만 볼 수 있다.
- 02 학습에서 모든 위치를 병렬로 계산해도 이 규칙을 지킨다.
- 03 미래 점수에 $-\infty$ 를 넣고 softmax한다.

05-20분

위치 정보가 필요한 이유

- 01 내용만 같으면 순서 차이를 직접 표시하기 어렵다.
- 02 토큰 embedding에 위치 embedding을 더한다.
- 03 문맥 길이는 위치 embedding 범위를 넘지 않아야 한다.

$$H_t = \text{token_embedding}(x_t) + \text{position_embedding}(t)$$

05-20분

전체 식을 세 단계로 읽기

- 01 QK^T 로 query와 key의 내적을 계산한다.
- 02 $\sqrt{d_k}$ 로 나누고 각 행을 softmax한다.
- 03 행의 가중치로 V 를 합한다.

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k} + M)V$$

20-35분

shape 추적표

01 Q,K,V는 [B,T,D]다.

02 $Q @ K.transpose(-2,-1)$ 은 [B,T,T]다.

03 가중치 @ V의 결과는 [B,T,D]다.

$$[B,T,D] \times [B,D,T] \rightarrow [B,T,T]$$

20-35분

손계산 · 두 key와 한 query

01 $q=[1,0]$, $k_1=[1,0]$, $k_2=[0,1]$ 로 둔다.

02 scaled score는 $[0.707,0]$ 이다.

03 softmax는 약 $[0.670,0.330]$ 이다.

$$v_1=[2,0], v_2=[0,2] \rightarrow \text{output} \approx [1.34, 0.66]$$

20-35분

왜 $\text{sqrt}(d_k)$ 로 나눌까

- 01 독립·평균 0·분산 1인 성분을 가정한다.
- 02 d_k 개 곱의 합은 분산이 d_k 규모가 된다.
- 03 스케일을 줄여 softmax가 너무 뽕족해지는 것을 완화한다.

$$\text{Var}(q \cdot k) \approx d_k \rightarrow \text{Var}((q \cdot k) / \text{sqrt}(d_k)) \approx 1$$

20-35분

causal mask 손으로 채우기

- 01 길이 3이면 첫 행은 첫 key만 허용한다.
- 02 둘째 행은 첫째·둘째 key를 허용한다.
- 03 마지막 행은 세 key를 모두 허용한다.

$$M = [[0, -\infty, -\infty], [0, 0, -\infty], [0, 0, 0]]$$

20-35분

언어 모델 손실

- 01 logits는 $[B, T, \text{Vocab}]$ 이고 target은 $[B, T]$ 다.
- 02 B와 T를 묶어 위치별 cross entropy를 계산한다.
- 03 평균 손실의 지수값은 이 설정의 perplexity다.

`CE(logits.reshape(-1,V), target.reshape(-1))`

20-35분

한 헤드부터 직접 구현

token+position

single-head attention

Linear

next token

- 01 Linear 세 개로 Q·K·V를 만든다.
- 02 scaled score와 causal mask를 직접 계산한다.
- 03 attention 결과를 vocabulary logits로 변환한다.

35-50분

softmax 축을 검증하기

각 query 행의 가중치 합이 1인지 검사한다.

미래 key의 가중치가 0인지 검사한다.

출력 shape와 유한값 여부를 검사한다.

```
assert torch.allclose(A.sum(-1),  
torch.ones_like(A.sum(-1)))
```

35-50분

코퍼스를 먼저 분할하기

- 01 문장 단위 train/validation/test를 먼저 나눈다.
- 02 각 split 안에서만 문자 시퀀스와 window를 만든다.
- 03 겹치는 window를 무작위로 나눈 뒤 평가하지 않는다.

문장 분할 → 인코딩 → split별 window

35-50분

입력과 target 확인

- 01 block_size=32면 33개 토큰이 필요하다.
- 02 앞 32개는 입력, 뒤 32개는 정답이다.
- 03 decode로 한 칸 shift를 눈으로 확인한다.

```
x=tokens[i:i+T], y=tokens[i+1:i+T+1]
```

35-50분

샘플링은 선택 규칙이다

- 01 `argmax`는 가장 높은 확률을 항상 고른다.
- 02 `multinomial`은 분포에서 무작위로 뽑는다.
- 03 `temperature`는 `logits`를 나눠 분포의 집중도를 바꾼다.

$$p = \text{softmax}(\text{logits} / \text{temperature}), \text{temperature} > 0$$

35-50분

LAB 1 · attention 연산 구현

- 01 practice §1–3: Q·K·V 손계산과 코드를 비교한다.
- 02 mask 전후 heatmap을 나란히 그린다.
- 03 행 합·미래 차단·shape assert를 통과한다.

50–58분 / 산출물: 점수와 attention heatmap

50–75분 · practice.ipynb와 함께 진행

LAB 2 · 문자 언어 모델 학습

- 01 practice §4–5: 코퍼스 분할과 shift를 확인한다.
- 02 bigram과 단일 헤드 모델을 짧게 학습한다.
- 03 동일 validation 토큰에서 손실을 비교한다.

58–70분 / 산출물: CE·perplexity 비교

50–75분 · practice.ipynb와 함께 진행

LAB 3 · 생성과 누출 검사

- 01 practice §6–7: temperature를 바꾸어 생성한다.
- 02 미래 토큰만 바뀌 이전 logits가 동일한지 검사한다.
- 03 학습된 attention 한 행을 읽고 한계를 적는다.

70–75분 / 산출물: 생성 3개 + causal 검사

50–75분 · practice.ipynb와 함께 진행

좋은 문자열과 좋은 모델

- 01 그럴듯한 샘플 한 개는 충분한 평가가 아니다.
- 02 held-out loss와 여러 prompt 출력을 함께 본다.
- 03 반복·철자·문장 구조를 분리해 기록한다.

숫자 평가 + 표본 관찰 + 데이터 한계

75-85분

mask를 빼면 왜 손실이 낮을까

- 01 학습 target은 입력의 다음 위치에 존재한다.
- 02 미래 참조를 허용하면 정답을 훔쳐볼 수 있다.
- 03 생성 시 미래가 없으므로 평가와 사용 조건이 달라진다.

낮은 손실이 항상 좋은 모델은 아니다

75-85분

attention 그림의 한계

- 01 가중치는 value를 섞는 계수다.
- 02 뒤의 Linear와 다른 표현 경로도 예측에 영향을 준다.
- 03 높은 가중치를 인과적 중요도로 단정하지 않는다.

관계를 관찰하되 설명을 과장하지 않는다

75-85분

출구 퀴즈

- 01 Q1. QK^T 가 만드는 두 T축은 각각 무엇인가?
- 02 Q2. mask는 softmax 전과 후 중 언제 적용할까?
- 03 Q3. value 대신 key를 가중합하면 같은 연산일까?

query 위치 / key 위치 / 전달할 내용

85-90분

다음: 여러 관점과 깊이

- 01 단일 헤드로 한 종류의 혼합을 만들었다.
- 02 여러 헤드와 FFN으로 표현을 확장한다.
- 03 residual과 LayerNorm으로 블록을 연결한다.

자율 과제: mask-scaled attention·문맥 길이

85-90분

부록 · 안정적인 softmax

- 01 각 행에서 최댓값을 빼도 확률은 같다.
- 02 지수 계산의 overflow 위험을 줄인다.
- 03 PyTorch softmax는 직접 exp를 나누는 것보다 적절하다.

$$\text{softmax}(s) = \text{softmax}(s - \max(s))$$

수업 외 심화

부록 · padding mask

- 01 길이가 다른 문장을 배치로 묶을 때 padding을 넣는다.
- 02 padding key를 보지 않도록 차단할 수 있다.
- 03 padding target은 손실에서 ignore_index로 제외한다.

causal mask + padding key mask + loss mask

수업 외 심화

부록 · API의 boolean mask

- 01 직접 masked_fill은 True인 곳을 차단하도록 작성할 수 있다.
- 02 SDPA의 bool attn_mask는 True인 곳을 허용한다.
- 03 MultiheadAttention의 일부 mask 규칙과 혼동하지 않는다.

사용 중인 API 문서를 먼저 확인한다

수업 외 심화

부록 · 문맥 길이와 비용

- 01 attention 행렬은 $T \times T$ 다.
- 02 문맥 길이를 두 배로 하면 원소 수는 네 배다.
- 03 projection 비용과 batch 크기도 총비용에 영향을 준다.

$$\text{attention matrix memory} \propto B T^2$$

수업 외 심화

부록 · tokenizer의 선택

- 01 문자 단위는 vocabulary가 작고 시퀀스가 길다.
- 02 단어 단위는 희귀 단어와 UNK 문제가 커진다.
- 03 subword는 두 방식 사이의 절충을 제공한다.

토큰화가 바뀌면 T와 V가 함께 바뀐다

수업 외 심화

부록 · 온도 손계산

- 01 logits=[2,1,0]에서 τ 를 비교한다.
- 02 $\tau=0.5$ 는 더 뾰족하고 $\tau=2$ 는 더 완만하다.
- 03 τ 가 작아도 확률 샘플링은 완전한 argmax와 다를 수 있다.

$\tau \rightarrow 0^+$ 일 때 최대 점수에 확률 집중

수업 외 심화

부록 · baseline의 가치

- 01 bigram은 장거리 정보를 볼 수 없다.
- 02 구현이 단순해 데이터·loss 오류를 찾기 쉽다.
- 03 복잡한 모델이 이기지 못하면 설정과 예산을 점검한다.

baseline은 비교 기준이자 디버깅 도구다

수업 외 심화

시각 보충 · 미래를 가리는 삼각형

1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0
1	1	1	1	1

행 = query / 열 = key

1 = 허용 / 0 = 차단 (논리 mask)

softmax 후 각 행의 확률 합 = 1

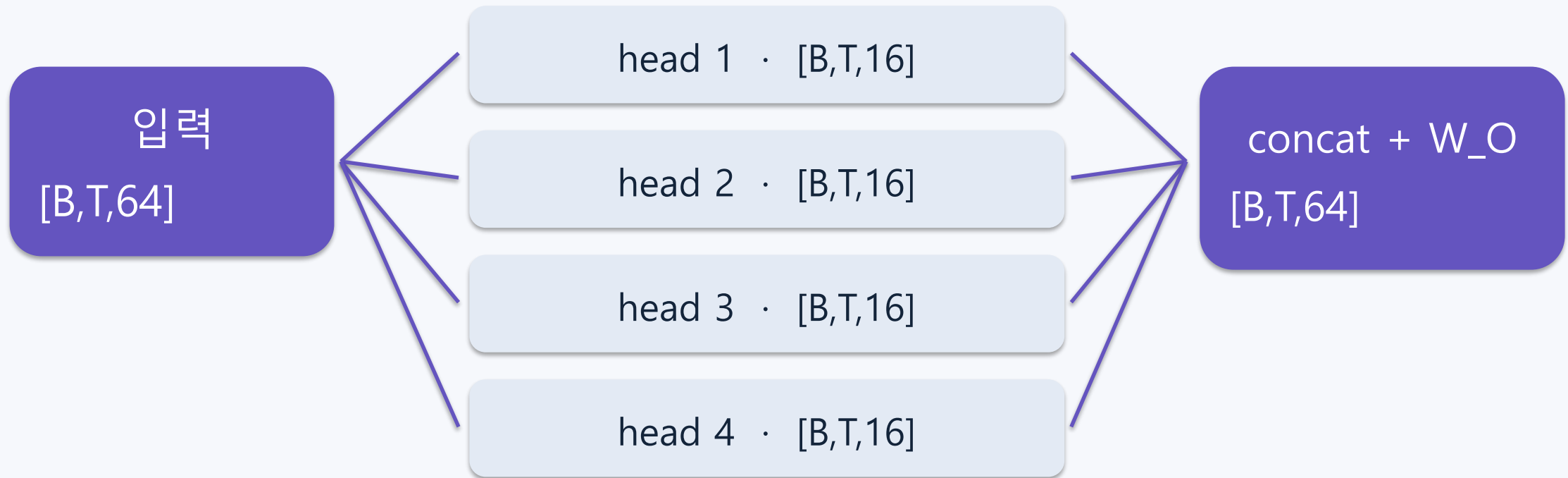
위치 2는 0·1·2만 참조한다.

입력의 현재 토큰은 허용하고

그 다음 토큰을 정답으로 예측한다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · head 분할과 결합



head 축을 복원할 때 $\text{transpose} \rightarrow \text{contiguous} \rightarrow \text{view}$ 순서를 확인한다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · next-token shift

원본

t	h	e		c	a	t
---	---	---	--	---	---	---

입력 x

t	h	e		c	a
---	---	---	--	---	---

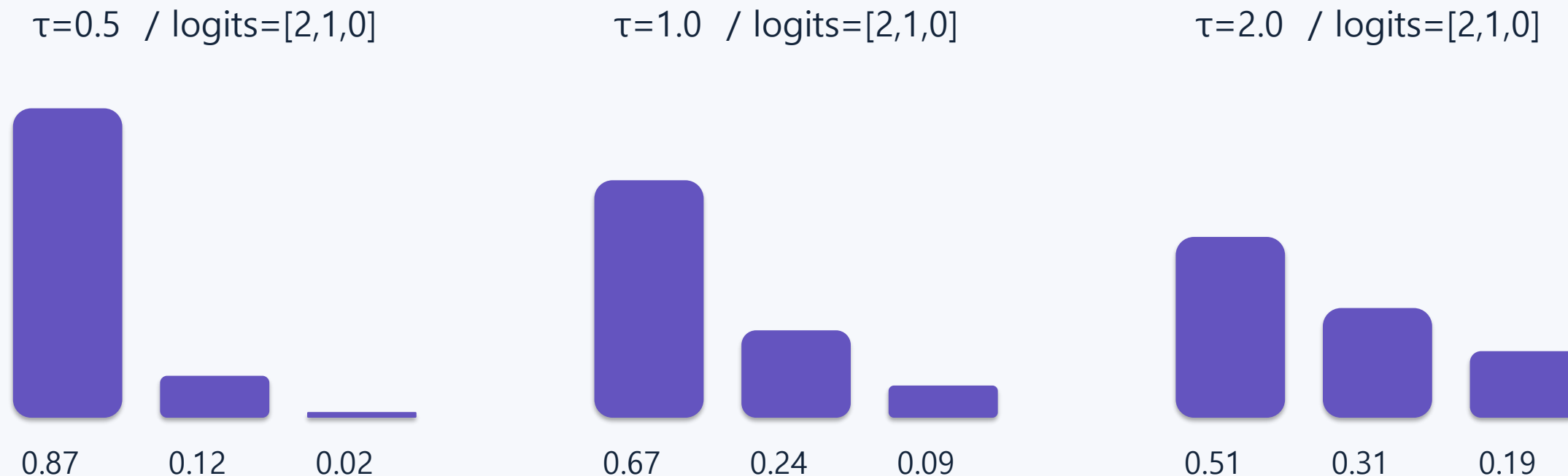
정답 y

h	e		c	a	t
---	---	--	---	---	---

같은 열에서
 $x_t \rightarrow y_t$
 정답은 원본의 다음 위치

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · temperature와 확률



설명용 확률 계산. 온도는 모델 가중치를 바꾸지 않고 선택 분포를 바꾼다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

읽을거리와 출처

- 01 Attention Is All You Need · Vaswani et al.
- 02 Scaled Dot Product Attention · 공식 API
- 03 CrossEntropyLoss · 공식 API

원문 링크는 발표자 노트와 주차별 README에 수록

생각하기 → 손으로 계산하기 → 코드로 검증하기